Vityarthi AIML Project

Course Code: *CSA2001*

Course Title: *Fundamentals of AIML*

Submitted By:

- **Name:** *Rahul Singh*
- **Reg No.:** *25BCE11302*
- **Branch:** *CSE (CORE)*
- **Year:** *1st Year*

Institution: *VIT Bhopal University*

Date: *30/03/2026*

Route Optimizer

Solving the Traveling Salesperson Problem using Simulated Annealing

1. Introduction and Problem Statement

In real-world logistics, delivery companies must find the shortest possible route to visit multiple locations and return to the starting warehouse. This is known in computer science as the Traveling Salesperson Problem (TSP). Because the number of possible routes grows factorially with each added destination, it is an NP-hard problem. Checking every single combination becomes computationally impossible for large datasets. This project solves this real-world business problem by building an AI agent that uses a heuristic search algorithm to find a highly optimized, near-perfect route in milliseconds.

2. AI Concept: Simulated Annealing

To solve this, the project utilizes **Simulated Annealing**, a stochastic (probabilistic) heuristic search algorithm. Basic AI algorithms, like "Hill Climbing," often fail at this task because they only accept moves that strictly improve the route. This causes the AI to get trapped in a "local minimum"—a route that looks good locally, but is not the best overall path.

Simulated Annealing solves this by mimicking the metallurgical process of heating and slowly cooling metal. When the algorithm's "temperature" is high, it is highly explorative and will actually accept *worse* routes. This allows the AI agent to jump out of local minimums and continue searching the state space for the true global optimum.

3. Mathematical Model and Logic

The AI evaluates its environment using a cost function based on Euclidean distance:

$$d = \sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]}$$

During the optimization loop, the AI randomly swaps two cities to create a neighboring state. If the new route is shorter, it is automatically accepted. If the new route is longer (a worse state), the AI decides whether to accept it based on the following probability function:

$$P = e^{(-\Delta E / T)}$$

Where:

- **P** is the probability of accepting the worse route.
- **ΔE** is the difference in distance between the new route and the current route.
- **T** is the current temperature of the system.

Because of how this mathematical function works, as T decreases over time (cooling), the probability P approaches zero. The AI stops acting randomly and strictly exploits the best path it has found.

4. Methodology and Implementation

The project is built entirely in Python using standard libraries to ensure a lightweight, CLI-based execution.

- **State Space:** All possible permutations of the delivery stops.
- **Initial State:** A randomly shuffled route.
- **State Transition:** Swapping two random stops in the list array using tuple unpacking.
- **Goal State:** A route where the total distance is minimized.

5. Conclusion

The CLI program successfully reads a dataset of coordinates, calculates a baseline random route, and applies the Simulated Annealing algorithm to compress the travel distance. By leveraging probabilistic acceptance, the AI agent consistently navigates a massive state space to output an optimal delivery sequence, demonstrating a clear, practical application of foundational Artificial Intelligence